AI Security

Optimization for Black-box Attacks

Sangdon Park

POSTECH

April 14, 2026

Outline

1. Introduction
2. Prompt Tuning
3. Zeroth Order Optimization
4. Policy Optimization

Need for Black-box Attacks

Main Question

How can we find an adversarial prompt in this black-box setup?

- Closed models (*e.g.*, GPT-5) is the most interesting attack targets
- However, we cannot have access to their model weights.
- But, still we can obtain responses from them (*i.e.*, only allows forward-pass).

Outline

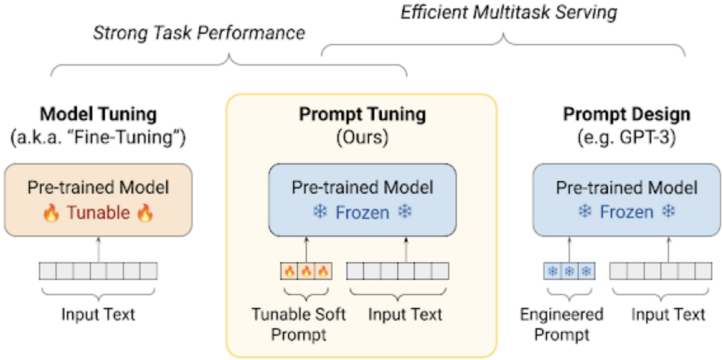
1 Introduction

2 Prompt Tuning

3 Zeroth Order Optimization

4 Policy Optimization

Why Prompt Tuning?



- Prompt tuning is a key to guide LLMs to have desired responses.
- We often build hard prompts for it.
- Prompt Tuning [Lester et al., 2021] finds soft input embedding vectors, instead of hard prompts.
- This is used for maximizing performance but can be applicable for attacks.

Prompt Tuning [Lester et al., 2021]

Objective

$$\min_{\mathbf{E}} -\sum_{n=1}^{N} \log p_\theta(\mathbf{y}_n \mid \mathbf{E}, \mathbf{x}_n)$$

- D: the embedding vector dimension
- K: the number of tuning tokens
- $\mathbf{E} \in \mathbb{R}^{D \times K}$: a set of input embedding vectors
- p_θ: an LLM
- In attacks where $\mathbf{y}_n$ is a harmful response, is it reasonable for attackers to manipulate $\mathbf{E}$?

Outline

1. Introduction
2. Prompt Tuning
3. Zeroth Order Optimization
4. Policy Optimization

What and Why Zeroth Order Optimization?

Question

Can we optimize parameters without gradients (or with only forward passes)?

- zeroth order optimization (ZO) methods optimizes a model by estimating gradients using forward passes.
- Why ZO?
  - no gradients – memory efficient
  - non-differentiable criteria, *e.g.*, rule-based safety standards or F1 score
- Why ZO in attacks?
  - may be useful for black-box models

Simultaneous Perturbation Stochastic Approximation (SPSA)

Definition (SPSA)

SPSA estimate the gradient of a loss function $\mathcal{L}$ for a model θ on a minibatch $\mathcal{B}$ as follows:

$$\hat{\nabla}\mathcal{L}(\theta;\mathcal{B}) = \frac{\mathcal{L}(\theta+\varepsilon z;\mathcal{B}) - \mathcal{L}(\theta-\varepsilon z;\mathcal{B})}{2\varepsilon} z \approx zz^T \nabla\mathcal{L}(\theta;\mathcal{B}),$$

where $z \sim \mathcal{N}(0, I_d)$ and ε is the perturbation scale.

- Intuitively, this estimates a gradient (*e.g.*, imagine then $d = 1$)
- How many forward passes do we need?
- How much memory do we need?
- Is this unbiased estimator?

MeZO [Malladi et al., 2023]

Algorithm 1: MeZO

Require: parameters $\boldsymbol{\theta} \in \mathbb{R}^d$, loss $\mathcal{L} : \mathbb{R}^d \to \mathbb{R}$, step budget T, perturbation scale ϵ, batch size B, learning rate schedule $\{\eta_t\}$

for $t = 1, ..., T$ **do**
  Sample batch $\mathcal{B} \subset \mathcal{D}$ and random seed s
  $\boldsymbol{\theta} \leftarrow$ `PerturbParameters(`$\boldsymbol{\theta}, \epsilon, s$`)`
  $\ell_+ \leftarrow \mathcal{L}(\boldsymbol{\theta}; \mathcal{B})$
  $\boldsymbol{\theta} \leftarrow$ `PerturbParameters(`$\boldsymbol{\theta}, -2\epsilon, s$`)`
  $\ell_- \leftarrow \mathcal{L}(\boldsymbol{\theta}; \mathcal{B})$
  $\boldsymbol{\theta} \leftarrow$ `PerturbParameters(`$\boldsymbol{\theta}, \epsilon, s$`)` ▷ `Reset parameters before descent`
  `projected_grad` $\leftarrow (\ell_+ - \ell_-)/(2\epsilon)$
  Reset random number generator with seed s ▷ `For sampling` z
  for $\theta_i \in \boldsymbol{\theta}$ **do**
    $z \sim \mathcal{N}(0, 1)$
    $\theta_i \leftarrow \theta_i - \eta_t *$ `projected_grad` $* z$
  end
end

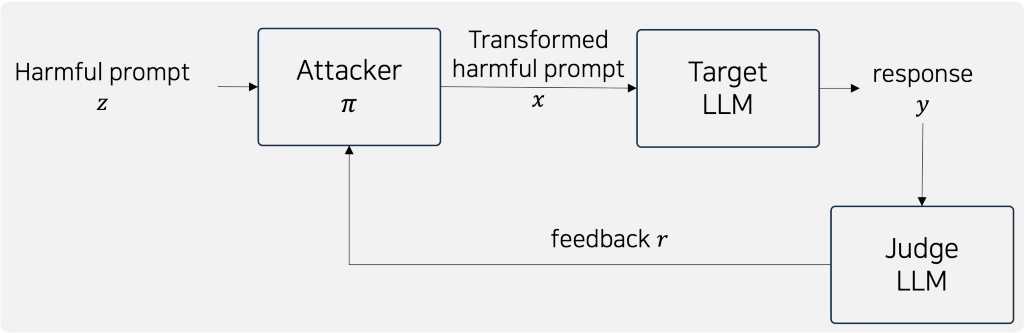
Subroutine `PerturbParameters(`$\boldsymbol{\theta}, \epsilon, s$`)`
  Reset random number generator with seed s ▷ `For sampling` z
  for $\theta_i \in \boldsymbol{\theta}$ **do**
    $z \sim \mathcal{N}(0, 1)$
    $\theta_i \leftarrow \theta_i + \epsilon z$ ▷ `Modify parameters in place`
  end
  return $\boldsymbol{\theta}$

- MeZO: Memory-efficient Zeroth order Optimizer
- Need only $\mathcal{O}(\mathbb{R}^d)$ memory

Outline

1. Introduction
2. Prompt Tuning
3. Zeroth Order Optimization
4. Policy Optimization

Why Policy Optimization?

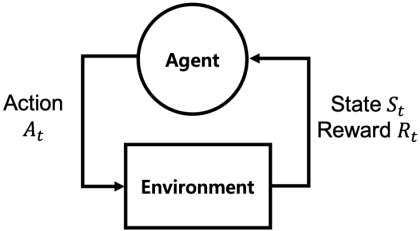


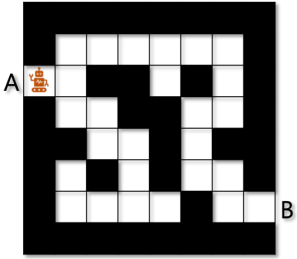
- We often see only the victim model's responses – black box setups
- Finding a good attacker = learning an attacker via RL – very natural

Learning from Interaction

Learning protocol:

- The agent observes an initial state S_0
- For each step $t = 0, 1, 2, \ldots$
- The agent executes an action A_t given the observed state S_t
- The agent observes reward R_{t+1} and the next state S_{t+1}





Markov Decision Process (MDP)

Definition (informal)

A Markov decision process (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p, d_0)$, where

- $\mathcal{S}$: a set of states
- $\mathcal{A}$: a set of actions (or $\mathcal{A}(s)$: a set of actions given a state s)
- $\mathcal{R}$: a set of reward values
- $p : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S} \times \mathcal{R})$: dynamics or a kernel
- d_0: an initial state distribution

For example,

- $s_0 \sim d_0$: an initial location of a robot
- $a_0 \in \mathcal{A}$: "move to the right"
- $p(s_1, r_1 \mid s_0, a_0)$: a proability distribution over next states and rewards

Markov Decision Process (MDP)

Definition (informal)

A Markov decision process (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p, d_0)$, where

- $\mathcal{S}$: a set of states
- $\mathcal{A}$: a set of actions (or $\mathcal{A}(s)$: a set of actions given a state s)
- $\mathcal{R}$: a set of reward values
- $p : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S} \times \mathcal{R})$: dynamics or a kernel
- d_0: an initial state distribution

- In MDP, p completely characterizes the environment's dynamcis due to the Markov property.
  - See the next slide.

Markov Property

Definition

The dyanmics $p : (\mathcal{S} \times \mathcal{A})^* \to \Delta(\mathcal{S} \times \mathcal{R})$ has a Markov property if

$$\overbrace{\mathbb{P}\{S_{t+1} = s_{t+1}, R_{t+1} = r_{t+1} \,|\, S_{1:t} = s_{1:t}, A_{1:t} = a_{1:t}\}}^{p(s_{t+1}, r_{t+1} | s_{1:t}, a_{1:t})} = \underbrace{\mathbb{P}\{S_{t+1} = s_{t+1}, R_{t+1} = r_{t+1} \mid S_t = s_t, A_t = a_t\}}_{p(s_{t+1}, r_{t+1} | s_t, a_t)}.$$

- This means S_t include perfect information on previous action-environment interaction.
- Here, we assume that the dynamcis does not change over time t (=homogeneous Markov chain).

RL Goal

Goal – informal

The goal of RL is to find an optimal "policy" that maximizes an "expected cumulative reward" (or an "expected return").

Note that

- the goal is NOT maximizing immediate rewards.
- the expected return $\approx$ an estimate of rewards that the agent will recieve in future.

We will see the answers of the following questions:

- What is the "policy"?
- What is the "expected return"?

Policy

Definition

A policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ is a mapping from states to probabilities of selecting each possible action.

- The policy can be deterministic – in this case we simply denote it by $\pi : \mathcal{S} \to \mathcal{A}$.
- The policy is modeled via a neural net – this is our target to learn

Example:

	s_1	s_2	s_3
$\pi_1(a_i \| s_i)$	a_1 a_2 a_3 1.0 0 0	a_1 a_2 a_3 0.5 0.5 0	a_1 a_2 a_3 0.2 0.3 0.4
$\pi_2(a_i \| s_i)$	a_1 a_2 a_3 0 1.0 0	a_1 a_2 a_3 1/3 1/3 1/3	a_1 a_2 a_3 0 0.5 0.5

Trajectory

Trajectory

A trajectory τ is a sequence of states, actions, and rewards that is obtained by the interaction of a policy π with its environment, *i.e.*,

$$\tau := (S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, \ldots).$$

- Often called a roll-out
- Is the trajectory a random variable? If so, where are sources of randomness?

Return

Definition

We say that G_t is a return with a finite horizon T, defined as

$$G_t := R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T$$

where R_k is a reward at time k and γ is a discount factor.

- R_k: the immediate reward
- The discount factor quantifies the appreciation on future rewards.
  - $\gamma = 0$: only consider the immediate reward but ignore all future rewards.
  - $0 \leq \gamma \leq 1$: consider the immediate reward and discount the appreciation more on more "distant" future rewards.
  - $\gamma = 1$: equallty appreciate all future rewards, including the immediate reward.
- The return has a recursive structure, *i.e.*,

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

RL Goal

Goal – informal

The goal of RL is to find an optimal policy that maximizes an expected return, *i.e.*,

$$\max_{\pi} \mathbb{E} G_0.$$

- The expectation is taken over trajectories.
- How to find the optimal policy?
  - Policy optimization
  - "Q" function and "V" function

Reference I

B. Lester, R. Al-Rfou, and N. Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, pages 3045–3059, 2021.

S. Malladi, T. Gao, E. Nichani, A. Damian, J. D. Lee, D. Chen, and S. Arora. Fine-tuning language models with just forward passes. *Advances in Neural Information Processing Systems*, 36:53038–53075, 2023.